



Trabalho prático

O trabalho pode ser feito em grupos de **até 5** alunos. As implementações podem ser feitas nas linguagens **Java ou Python**, e o relatório deve ser feito em **L^AT_EX**. Ele deverá ser entregue no Canvas, até as 23:59 da data limite estipulada (data mostrada na tarefa do Canvas). Cópias serão sumariamente zeradas. A entrega deve ser realizada da seguinte forma: **um** arquivo *.zip* ou *.rar* contendo o código fonte do trabalho, o fonte do relatório (*.tex*) e o PDF do relatório. Além disso, indiquem no relatório as responsabilidades e o que foi feito por cada membro do grupo. As entregas intermediárias devem ser feitas através do Canvas, Github Classroom ou Verde, conforme definido nos enunciados de cada ponto de controle.

Trabalho Prático

Objetivo

Desenvolver uma biblioteca computacional para criação, manipulação e análise básica de grafos. A biblioteca deverá implementar uma API comum para grafos, permitindo que uma mesma aplicação utilize diferentes representações internas, como matriz de adjacência e lista de adjacência. Além disso, desenvolver uma aplicação utilizando a biblioteca desenvolvida, resolvendo um problema da vida real modelando-o utilizando grafos.

Etapa 1 – Construção da Biblioteca de Manipulação de Grafos

Nesta etapa, os alunos deverão implementar uma biblioteca para manipulação de grafos simples e direcionados. A biblioteca deverá permitir a criação de grafos, inserção e remoção de arestas, consulta de relações entre vértices, cálculo de graus, definição de pesos e exportação do grafo para visualização em ferramentas externas.

A implementação deverá ser organizada de forma que a lógica comum entre diferentes representações de grafos seja compartilhada por meio de uma classe abstrata, enquanto os detalhes específicos de cada representação sejam implementados em classes concretas.

Requisitos da Biblioteca

A biblioteca deverá conter, obrigatoriamente, a seguinte estrutura de classes:

- **Classe abstrata AbstractGraph:**
 - Deve definir a API comum da biblioteca;
 - Deve conter atributos compartilhados entre as implementações, como pesos dos vértices;
 - Deve conter métodos auxiliares, como verificação de índices válidos;

- Deve declarar os métodos abstratos que serão implementados pelas classes concretas.
- **Classe concreta `AdjacencyMatrixGraph`:**
 - Deve implementar a API utilizando **matriz de adjacência**;
 - Deve armazenar as arestas e seus pesos em estruturas baseadas em matrizes;
 - Deve respeitar todas as regras definidas pela classe abstrata.
- **Classe concreta `AdjacencyListGraph`:**
 - Deve implementar a API utilizando **listas de adjacência**;
 - Deve armazenar as arestas e seus pesos em estruturas baseadas em listas, dicionários, mapas ou estruturas equivalentes;
 - Deve respeitar todas as regras definidas pela classe abstrata.

Construtores Obrigatórios

As classes concretas deverão possuir construtores que recebam o número de vértices do grafo:

- `AdjacencyMatrixGraph(int numVertices)`
- `AdjacencyListGraph(int numVertices)`

O número de vértices deve ser definido no momento da criação do grafo. Os vértices serão identificados por índices inteiros, iniciando em 0 e terminando em `numVertices - 1`.

API Obrigatória

A biblioteca deverá implementar os seguintes métodos:

- `int getVertexCount();`
- `int getEdgeCount();`
- `boolean hasEdge(int u, int v);`
- `void addEdge(int u, int v);`
- `void removeEdge(int u, int v);`
- `boolean isSuccessor(int u, int v);`
- `boolean isPredecessor(int u, int v);`
- `boolean isDivergent(int u1, int v1, int u2, int v2);`
- `boolean isConvergent(int u1, int v1, int u2, int v2);`
- `boolean isIncident(int u, int v, int x);`
- `int getVertexInDegree(int u);`
- `int getVertexOutDegree(int u);`
- `void setVertexWeight(int v, double w);`
- `double getVertexWeight(int v);`
- `void setEdgeWeight(int u, int v, double w);`
- `double getEdgeWeight(int u, int v);`

- `boolean isConnected();`
- `boolean isEmptyGraph();`
- `boolean isCompleteGraph();`
- `void exportToGEPHI(String path);`

Descrição dos Métodos

Os métodos devem seguir as seguintes definições:

- `getVertexCount()`: retorna a quantidade de vértices do grafo.
- `getEdgeCount()`: retorna a quantidade de arestas existentes no grafo.
- `hasEdge(u, v)`: verifica se existe uma aresta direcionada de `u` para `v`.
- `addEdge(u, v)`: adiciona uma aresta direcionada de `u` para `v`.
- `removeEdge(u, v)`: remove a aresta direcionada de `u` para `v`, caso exista.
- `isSuccessor(u, v)`: verifica se `v` é sucessor de `u`.
- `isPredecessor(u, v)`: verifica se `u` é predecessor de `v`.
- `isDivergent(u1, v1, u2, v2)`: verifica se duas arestas são divergentes, ou seja, se possuem o mesmo vértice de origem.
- `isConvergent(u1, v1, u2, v2)`: verifica se duas arestas são convergentes, ou seja, se possuem o mesmo vértice de destino.
- `isIncident(u, v, x)`: verifica se o vértice `x` é incidente à aresta `(u, v)`.
- `getVertexInDegree(u)`: retorna o grau de entrada do vértice `u`.
- `getVertexOutDegree(u)`: retorna o grau de saída do vértice `u`.
- `setVertexWeight(v, w)`: define o peso do vértice `v`.
- `getVertexWeight(v)`: retorna o peso do vértice `v`.
- `setEdgeWeight(u, v, w)`: define o peso da aresta `(u, v)`.
- `getEdgeWeight(u, v)`: retorna o peso da aresta `(u, v)`.
- `isConnected()`: verifica se o grafo é conectado, considerando a conectividade de sua estrutura.
- `isEmptyGraph()`: verifica se o grafo não possui arestas.
- `isCompleteGraph()`: verifica se o grafo possui todas as arestas possíveis para um grafo simples direcionado, sem laços.
- `exportToGEPHI(path)`: exporta o grafo em um formato compatível com o software Gephi, como `.gexf`, `.graphml` ou `.csv`.

Restrições

A implementação deverá respeitar as seguintes restrições:

- Os grafos devem ser **simples**, ou seja, não devem permitir laços nem múltiplas arestas.
- Não deve ser permitido adicionar uma aresta do tipo (u, u) .
- O método `addEdge(u, v)` deve ser idempotente, isto é, chamadas repetidas com os mesmos vértices não devem criar múltiplas arestas.
- Devem ser lançadas exceções para índices de vértices inválidos.
- Devem ser lançadas exceções para operações inconsistentes, como consultar peso de uma aresta inexistente.
- É proibido o uso de bibliotecas prontas de grafos, como **NetworkX**, **JGraphT** ou similares.
- A biblioteca deve ser implementada pelos próprios alunos.
- O código deve estar organizado, comentado quando necessário e seguir boas práticas de programação.

Programa de Demonstração

Além da biblioteca, o grupo deverá desenvolver uma aplicação simples para demonstrar o uso da API implementada. Essa aplicação poderá ser executada em terminal e deverá mostrar, no mínimo:

- Criação de um grafo utilizando matriz de adjacência;
- Criação de um grafo utilizando lista de adjacência;
- Inserção e remoção de arestas;
- Consulta de sucessores e predecessores;
- Cálculo de grau de entrada e grau de saída;
- Definição e consulta de pesos de vértices;
- Definição e consulta de pesos de arestas;
- Verificação se o grafo é vazio;
- Verificação se o grafo é completo;
- Verificação se o grafo é conectado;
- Exportação do grafo para um formato compatível com o Gephi.

Etapa 2 – Modelagem e Resolução de um Problema Real com Grafos

Na segunda etapa do trabalho, o grupo deverá utilizar a biblioteca desenvolvida na Etapa 1 para modelar e resolver um problema do mundo real que possa ser representado por meio de grafos.

O grupo deverá escolher um problema que considere interessante, relevante ou desafiador, desde que seja possível representá-lo por meio de vértices, arestas, pesos e relações entre elementos. A escolha do problema é livre, mas deve ser justificada no relatório final.

Exemplos de problemas possíveis incluem, mas não se limitam a:

- Rotas entre cidades, bairros, pontos turísticos ou estações de transporte;
- Redes de amizade, colaboração, influência ou comunicação;

- Sistemas de recomendação baseados em relações entre usuários, produtos ou conteúdos;
- Fluxos de dependência entre tarefas, disciplinas, módulos de software ou etapas de um processo;
- Análise de conexões em redes sociais, redes de citações ou redes de coautoria;
- Modelagem de mapas, labirintos, jogos, cadeias produtivas ou redes de distribuição.

Definição do Problema

O grupo deverá apresentar claramente o problema escolhido, explicando:

- Qual é o problema do mundo real a ser estudado;
- Por que esse problema é interessante ou relevante;
- Quais decisões podem ser apoiadas pela análise do grafo;
- Quais dados serão utilizados para construir o grafo;
- Se os dados são reais, simulados ou construídos manualmente pelo grupo.

Não é obrigatório utilizar uma base de dados grande ou complexa. O mais importante é que a modelagem seja coerente e permita a aplicação dos algoritmos escolhidos.

Modelagem como Grafo

O grupo deverá propor uma modelagem do problema utilizando grafos. Essa modelagem deverá explicar:

- O que os vértices representam;
- O que as arestas representam;
- Se o grafo é direcionado ou não direcionado;
- Se as arestas possuem pesos e qual é o significado desses pesos;
- Se os vértices possuem pesos e qual é o significado desses pesos;
- Quais informações do problema real foram preservadas na modelagem;
- Quais simplificações ou limitações foram adotadas.

A construção do grafo deverá ser feita utilizando a biblioteca desenvolvida na Etapa 1. Portanto, a aplicação da Etapa 2 deve instanciar e manipular grafos por meio da API criada pelo próprio grupo.

Algoritmos da Literatura

O grupo deverá selecionar **dois algoritmos da literatura** relacionados a grafos e adequados ao problema escolhido. Esses algoritmos deverão ser estudados, descritos no relatório e implementados pelo grupo.

Exemplos de algoritmos possíveis incluem:

- Busca em largura;
- Busca em profundidade;
- Dijkstra;
- Bellman-Ford;
- Floyd-Warshall;

- Prim;
- Kruskal;
- Ordenação topológica;
- Detecção de componentes conexas;
- Algoritmos para caminho mínimo;
- Algoritmos para árvore geradora mínima;
- Algoritmos para centralidade, influência ou importância de vértices;
- Algoritmos para detecção de ciclos;
- Algoritmos para coloração de grafos.

A escolha dos algoritmos deve fazer sentido para o problema modelado. Por exemplo, em um problema de rotas, algoritmos de caminho mínimo podem ser adequados. Em um problema de dependência entre tarefas, ordenação topológica pode ser mais apropriada. Em um problema de rede de relações, medidas de centralidade ou componentes conexas podem ser úteis.

Algoritmos da Literatura

O grupo deverá selecionar **dois algoritmos da literatura** relacionados à análise de grafos, redes complexas, ranqueamento, propagação de informação, detecção de comunidades ou extração de métricas estruturais. Os algoritmos escolhidos devem estar associados a artigos científicos, livros ou materiais acadêmicos reconhecidos, e devem ser adequados ao problema modelado pelo grupo.

Nesta etapa, espera-se que os grupos procurem ir além dos algoritmos clássicos normalmente estudados em disciplinas introdutórias de grafos, como caminho mínimo, árvore geradora mínima ou coloração. Esses algoritmos podem até ser utilizados se forem muito bem justificados, mas a proposta principal desta etapa é incentivar a consulta à literatura e a implementação de métodos um pouco mais elaborados, usados em aplicações reais de análise de redes.

Exemplos de linhas de algoritmos e métodos que podem ser explorados incluem:

- **PageRank:** algoritmo de ranqueamento de vértices inspirado no funcionamento original de mecanismos de busca, no qual a importância de um vértice depende da importância dos vértices que apontam para ele. Pode ser aplicado em redes de citações, páginas, influenciadores, produtos, recomendações ou relações de dependência.
- **HITS:** algoritmo que calcula dois tipos de pontuação para os vértices, chamados de *hubs* e *authorities*. Pode ser usado para identificar vértices que apontam para boas fontes de informação e vértices que recebem conexões de boas fontes.
- **Métricas de centralidade em redes complexas:** métodos para identificar vértices importantes em uma rede, como centralidade de grau, centralidade de intermediação, centralidade de proximidade, centralidade de autovetor, centralidade de Katz ou variações dessas medidas.
- **Detecção de comunidades:** algoritmos que identificam grupos de vértices mais densamente conectados entre si do que com o restante da rede. Exemplos incluem métodos baseados em modularidade, remoção de arestas de alta intermediação, propagação de rótulos ou agrupamentos hierárquicos.
- **Coefficiente de agrupamento e transitividade:** métricas usadas para analisar o quanto os vizinhos de um vértice também se conectam entre si, permitindo estudar a formação de grupos locais em redes sociais, redes de colaboração ou redes de recomendação.
- **Assortatividade:** medida que analisa se vértices com características semelhantes tendem a se conectar entre si. Pode ser usada, por exemplo, para estudar redes sociais, redes acadêmicas, redes profissionais ou padrões de homofilia em redes.

- **Propagação de influência ou difusão em redes:** modelos que simulam como uma informação, opinião, comportamento, falha ou contaminação se espalha por uma rede. Podem ser usados modelos simples baseados em limiares, contágio independente ou propagação iterativa.
- **SimRank ou outras medidas de similaridade em grafos:** métodos que estimam a similaridade entre vértices a partir da estrutura da rede, podendo ser aplicados em recomendação, agrupamento, busca por perfis semelhantes ou análise de relações indiretas.
- **Deteção de pontes, vértices críticos ou vulnerabilidade da rede:** algoritmos que identificam vértices ou arestas cuja remoção altera significativamente a conectividade da rede. Podem ser usados para estudar robustez, pontos de falha, infraestrutura ou redes de dependência.
- **Análise espectral de grafos:** métodos baseados em autovalores e autovetores

Aplicação e Análise dos Resultados

Após implementar os algoritmos, o grupo deverá aplicá-los ao grafo construído para o problema escolhido.

A análise deverá discutir:

- Quais resultados foram obtidos;
- Como os resultados ajudam a compreender ou resolver o problema;
- Se os resultados fazem sentido em relação ao domínio escolhido;
- Quais limitações existem na modelagem, nos dados ou nos algoritmos;
- Que melhorias poderiam ser feitas em versões futuras do trabalho.

O grupo também deverá exportar o grafo utilizado para um formato compatível com o Gephi, permitindo sua visualização externa.

Documentação e Relatório Final

A entrega final deverá conter um relatório no formato de artigo técnico-científico, utilizando o modelo da SBC.

O relatório deverá conter, no mínimo:

- Introdução, apresentando o problema escolhido e sua motivação;
- Descrição da biblioteca desenvolvida na Etapa 1;
- Descrição da modelagem do problema como grafo;
- Fundamentação dos dois algoritmos escolhidos;
- Descrição da implementação;
- Apresentação e análise dos resultados;
- Discussão das limitações;
- Conclusão;
- Referências bibliográficas.

As referências devem incluir as fontes utilizadas para estudar os algoritmos escolhidos e, quando houver, as fontes dos dados utilizados no problema.

Entregáveis

Ao final do trabalho, o grupo deverá entregar:

- Código-fonte da biblioteca de grafos;
- Relatório final em PDF, no formato de artigo técnico-científico, utilizando o modelo da SBC, disponível em: <https://www.overleaf.com/latex/templates/sbc-conferences-template-updated-sbc-template-dot-sty-v2017/pyhttxftxjqn>